

Blind Signatures from Arguments of Inequality

Michael Klooß

KIT

Russell W. F. Lai

Aalto University

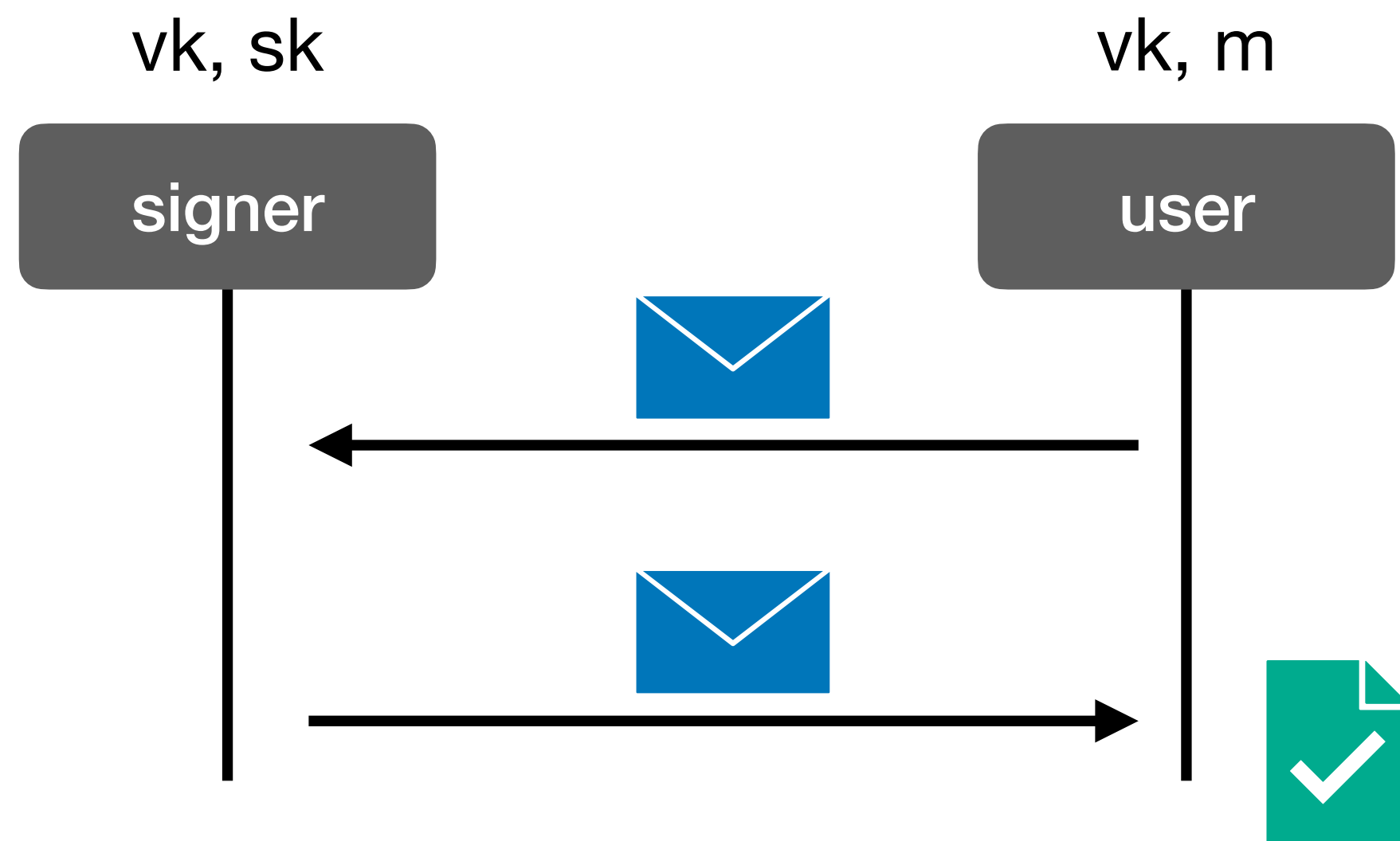
Michael Reichle

ETH Zurich



ETH zürich

Blind Signatures



Correctness:

- honest signatures  verify

Blindness:

- cannot link (, m) to the signing session

One-more Unforgeability: (OMUF)

- can only obtain ℓ valid (, m) pairs from ℓ concurrent signing sessions

Motivation

Pairing-free Groups in the ROM

- **Digital Signatures:** Schnorr is secure under DL [**Sch89,PS96**]
- **Known Tradeoffs for Blind Signatures:**

Limited Concurrency under DL:

- polylog-concurrency [**PS96, AO00, HKL19**]
- a-priori bound poly-concurrency [**KLR21, CHLLW22**]

In the AGM and ROM:

- under DDH [**Abe01, KLX22**] or mROS + DL [**FPS20**]
- under DL [**TZ22, CKMTZ23**]

- **Recent Progress:** CDH or DDH [**CTZ24, KRW24, KR25, BHKR25, RR26, Che26, CTR26**]

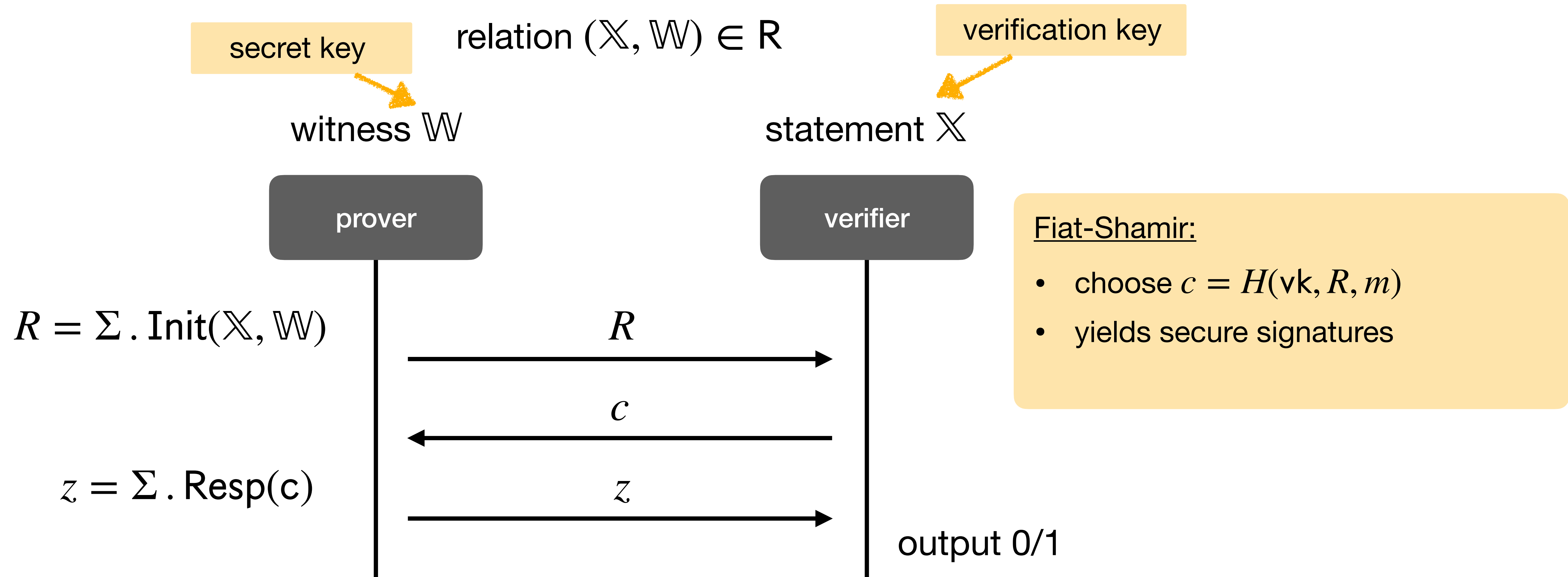
non-black box [**Fis06, FW24, MV26**]

Our Work in a Nutshell

- **Contribution:** Blind Signatures from DL in the ROM
 - no bound on concurrency
 - black-box use of group
 - no AGM / GGM
- **Technique:** based on proofs of inequality framework **[KR25]**
 - relies on statistical puncturing
 - here: computational puncturing + rewinding-based proof

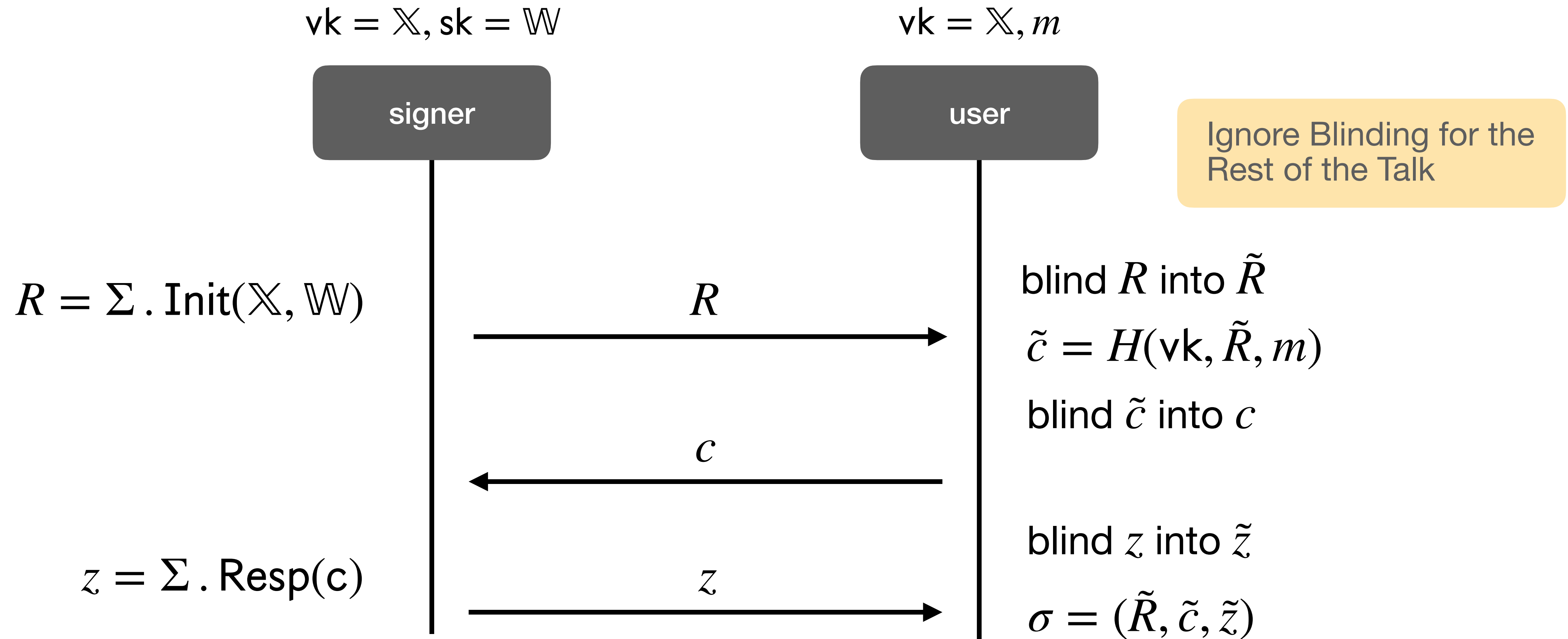
Design Paradigm

Σ -protocols and Fiat-Shamir



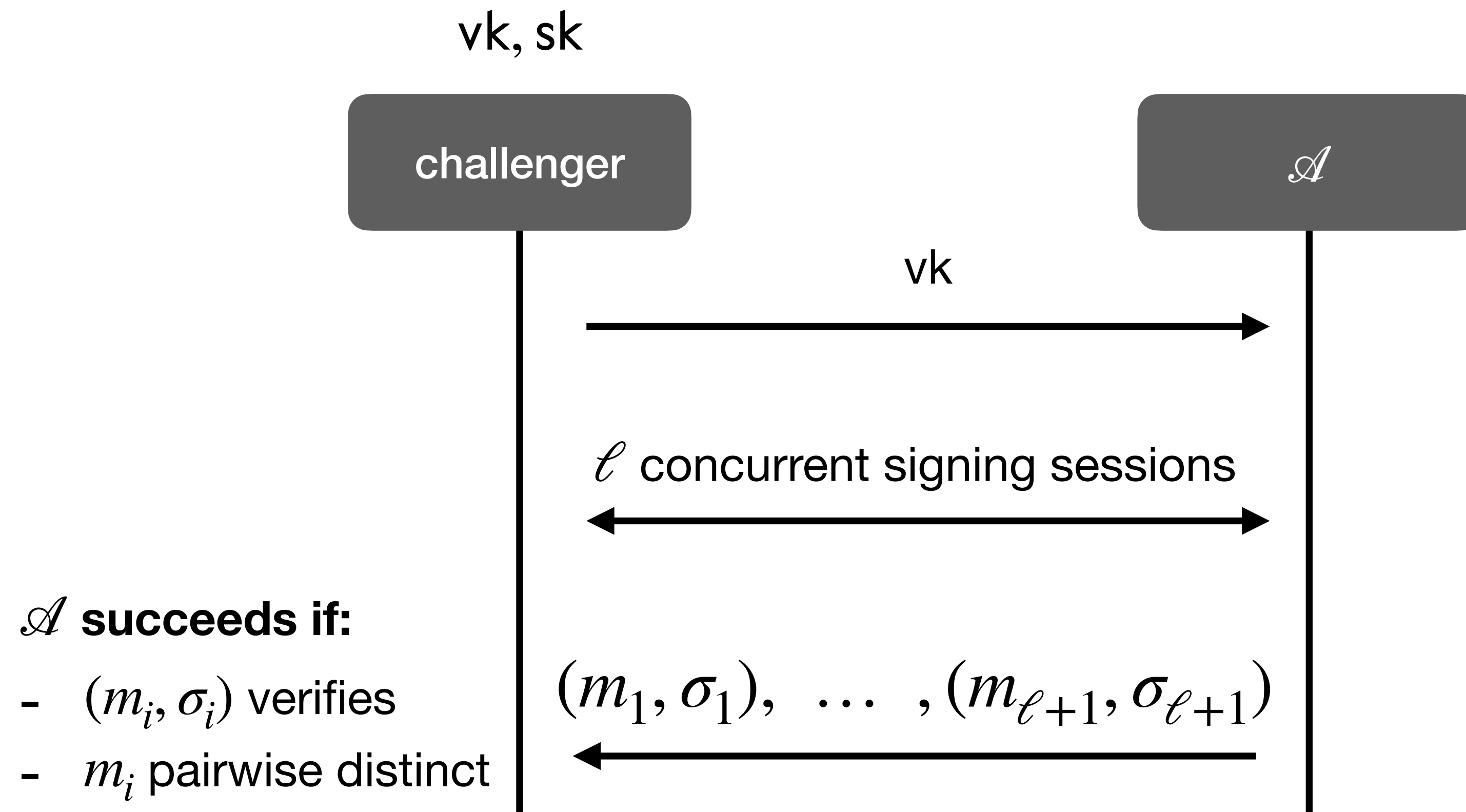
Design Paradigm

Generic Fiat-Shamir Blind Signatures



Fiat-Shamir Blind Signatures

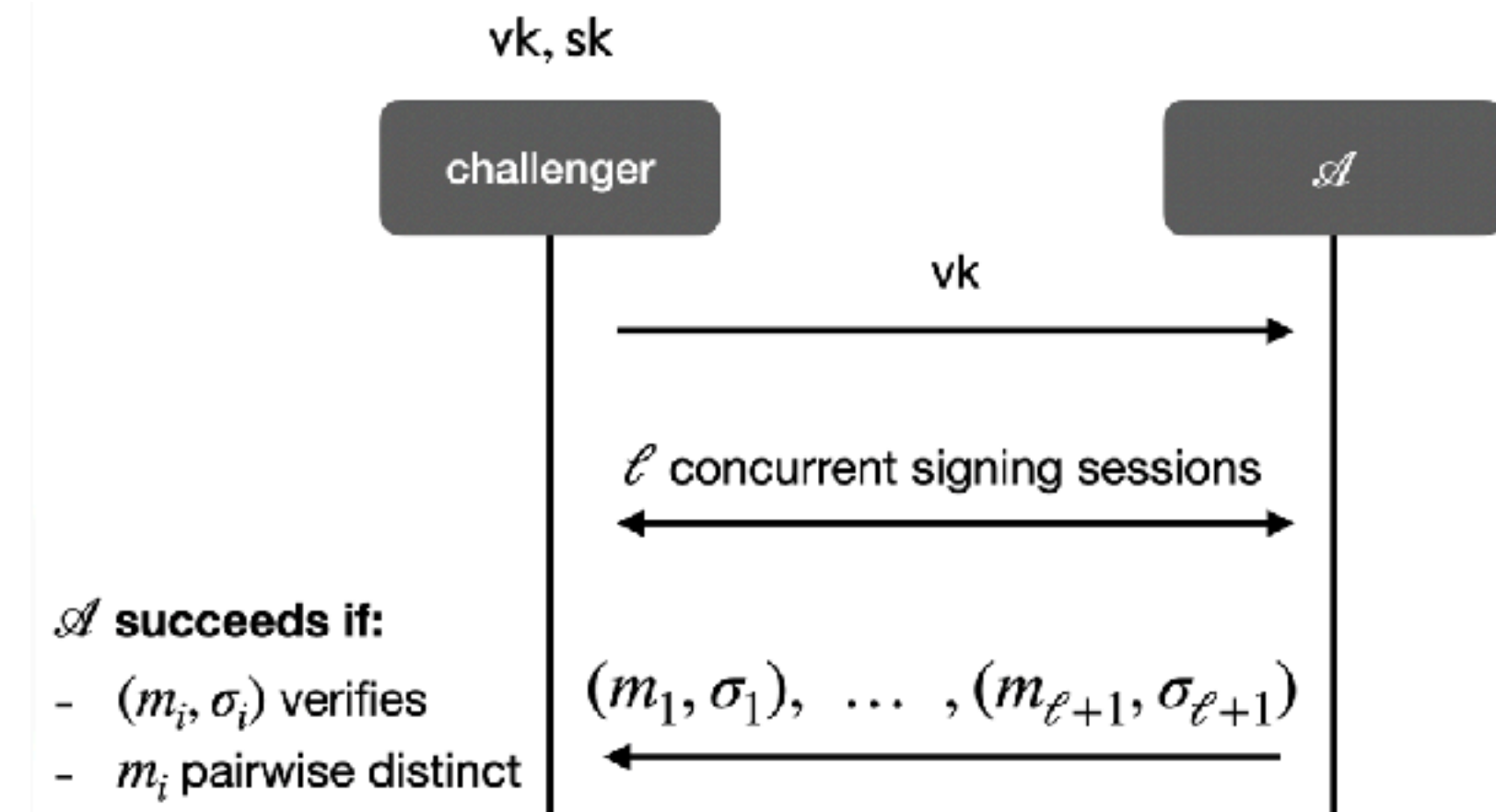
Challenge for Proving OMUF



Fiat-Shamir Blind Signatures

Challenge for Proving OMUF

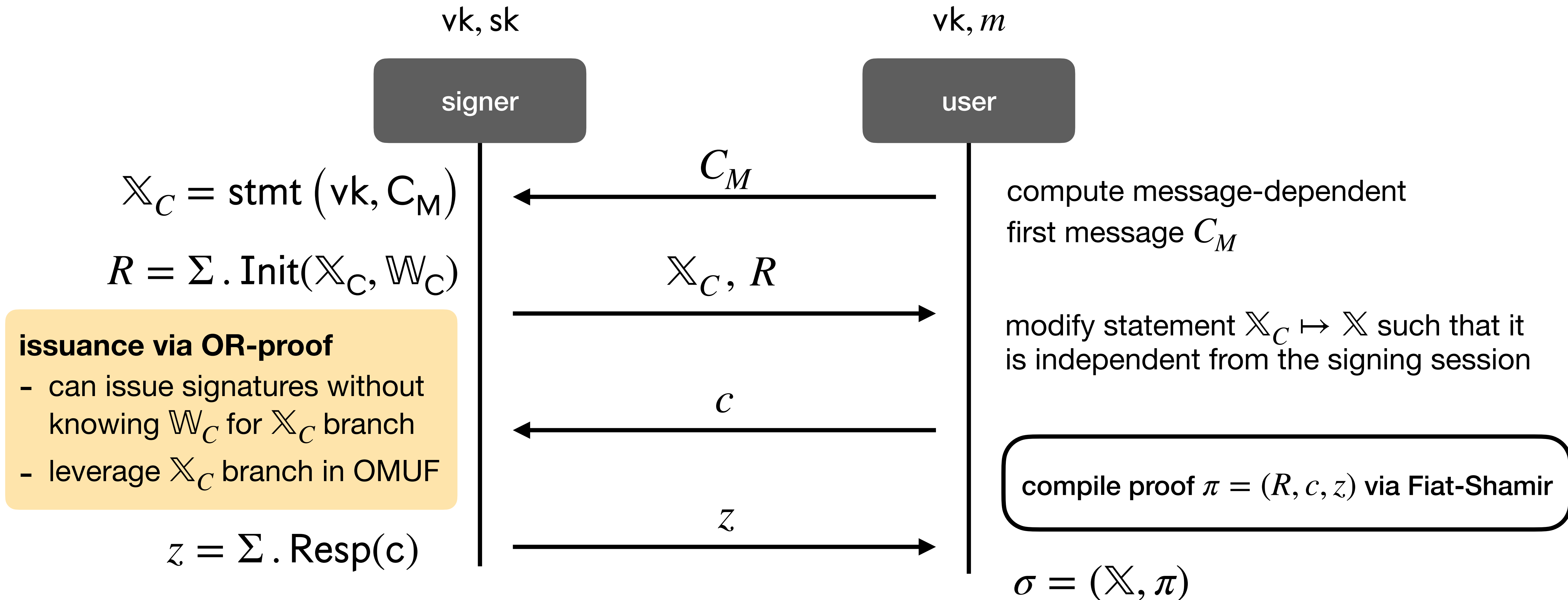
- **OMUF game simulates signing sessions:**
 - issues a valid Σ -protocol transcript for a random challenge c
 - must *know* a witness $\mathbb{W}$ for the statement $\mathbb{X}$
- **Subtle Problem:**
 - signature allows to extract *some* witness $\mathbb{W}$ for $\mathbb{X}$ via rewinding
 - idea: [PS96, AO00, HKL19]
 - design $\mathbb{X}$ with many witnesses
 - fresh witness $\mathbb{W}^*$ solves hard problem
 - issue: cannot argue that $\mathbb{W}^*$ must be fresh / distinct except for $\ell \leq \text{polylog}$



ROS Attack
[Sch01,BLLOR21]

Unlimited Concurrency in the ROM

Prior Works [CTZ24, KRW24, KR25, BHKR25, RR26]



Unlimited Concurrency in the ROM

Prior Works [CTZ24, KRW24, KR25, BHKR25, RR26]

- **OMUF:** design statements $\mathbb{X}_C$ and $\mathbb{X}$ are such that
 - simulator can compute a witness $\mathbb{W}_C$ for $\mathbb{X}_C$ in OMUF
 - one of $\mathcal{A}$'s signatures is hard to compute
 1. [CTZ24, KRW24, RR26]: signature's $\mathbb{X}$ contains a solution to a hard problem
 2. [KR25, BHKR25]: specific signature is *statistically*-hard to forge
 - Fiat-Shamir proof ensures well-formedness without rewinding

Remark: new 3-move schemes at Crypto'26 depart from this approach [CRT26, Che26]

Blind Signatures from DL

Blind Signatures from Puncturing

Recap of [KR25]

- **Technique:** craft statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) signer can sign all messages $m \neq m^*$
 - (ii) signature on m^* is hard to compute
 - $\mathbb{X}$ is compact and linear $\rightarrow$ efficient blind issuance
- **Statement $\mathbb{X}$:** inequality of encrypted messages

$C := C^* - \text{Enc}(\text{pk}, m; 0)$ does not encrypt 0

- **Puncturing:** encrypt m^* in C^*

ElGamal

Our Approach

Towards an instantiation from DL

- **Caveat of [KR25]:** relies on DDH for encryption
- **Idea:** replace ElGamal encryption with Pedersen commitments
- **Statement $\mathbb{X}$:** inequality of committed messages

$C := C^* - \text{Com}(m; 0)$ does not commit to 0

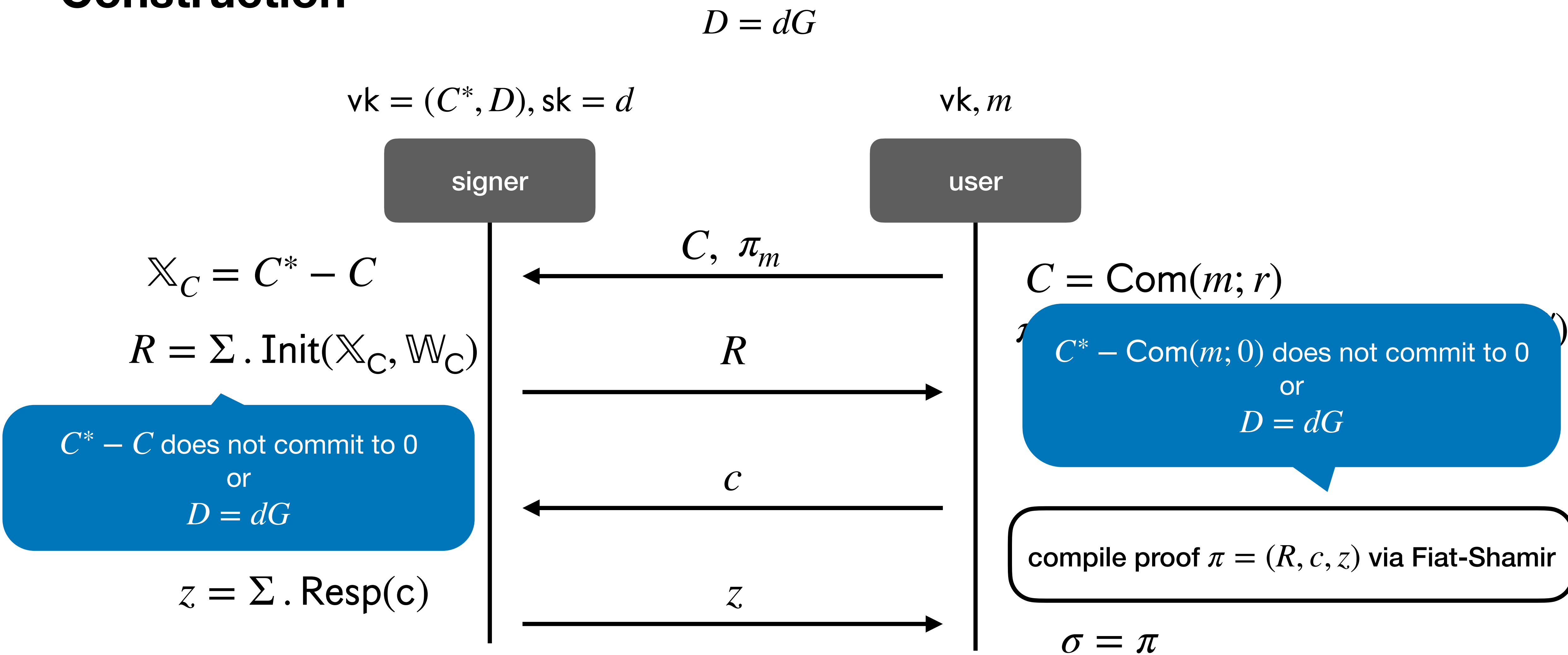
→ hides message m statistically, binds m under DL assumption

→ puncturing is only *computational*

hence: arguments of inequality

Our Approach

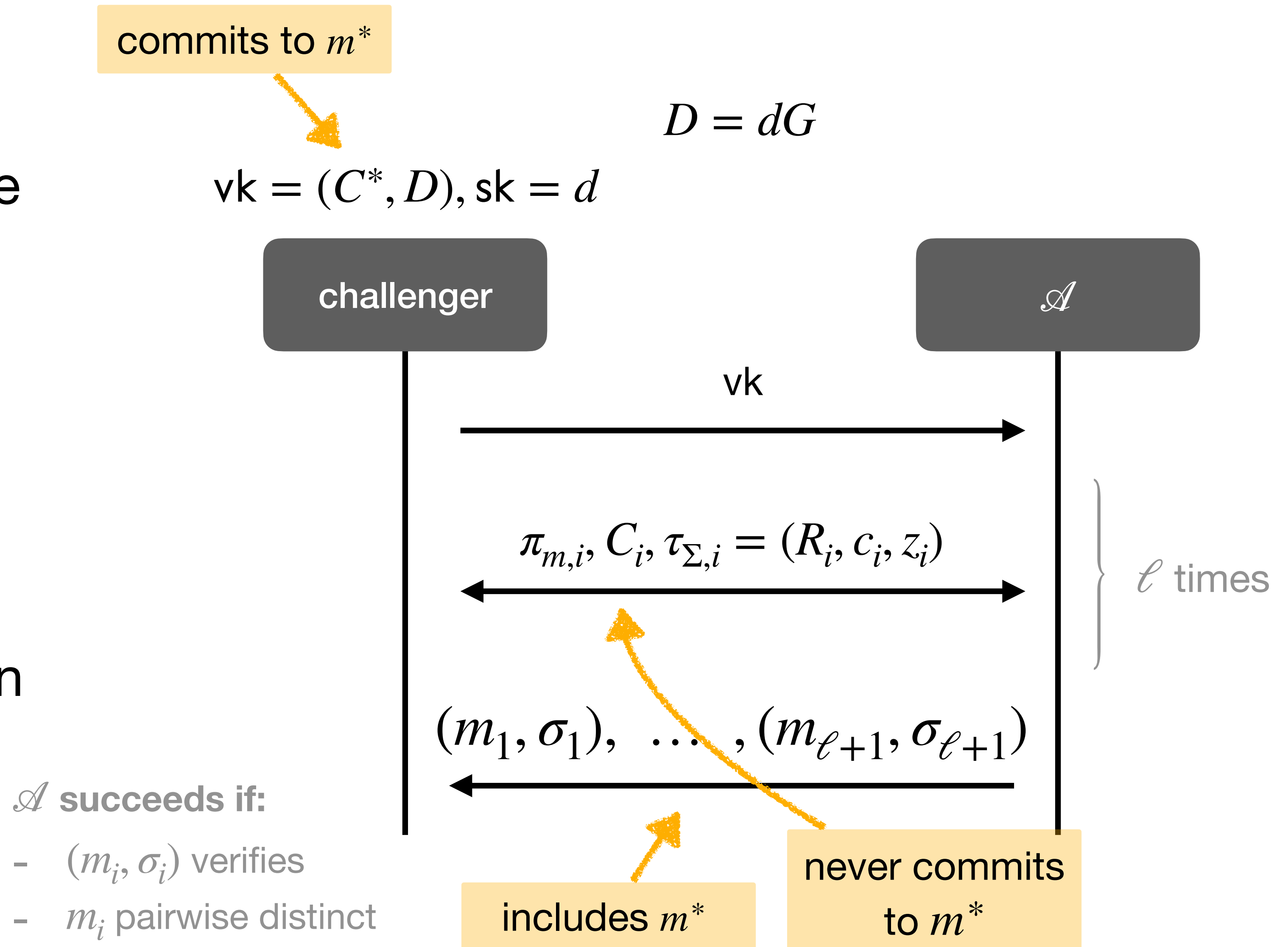
Construction



Our Approach

Unforgeability

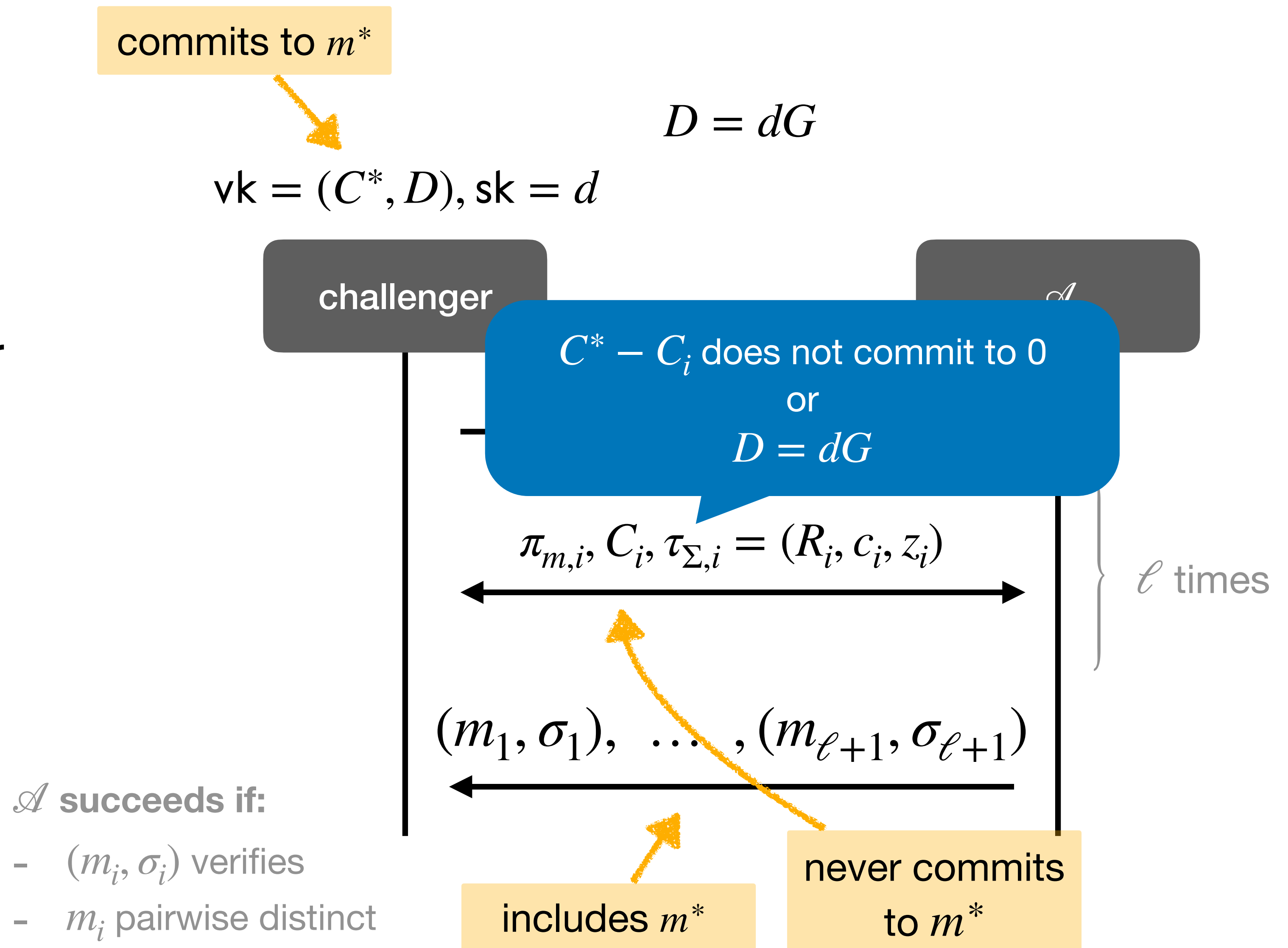
- **Step 1:** puncture vk on some message m^*
 - commit to m^* in C^*
 - force adversary to provide forgery for m^*
 - never sign m^* in simulation



Our Approach

Unforgeability

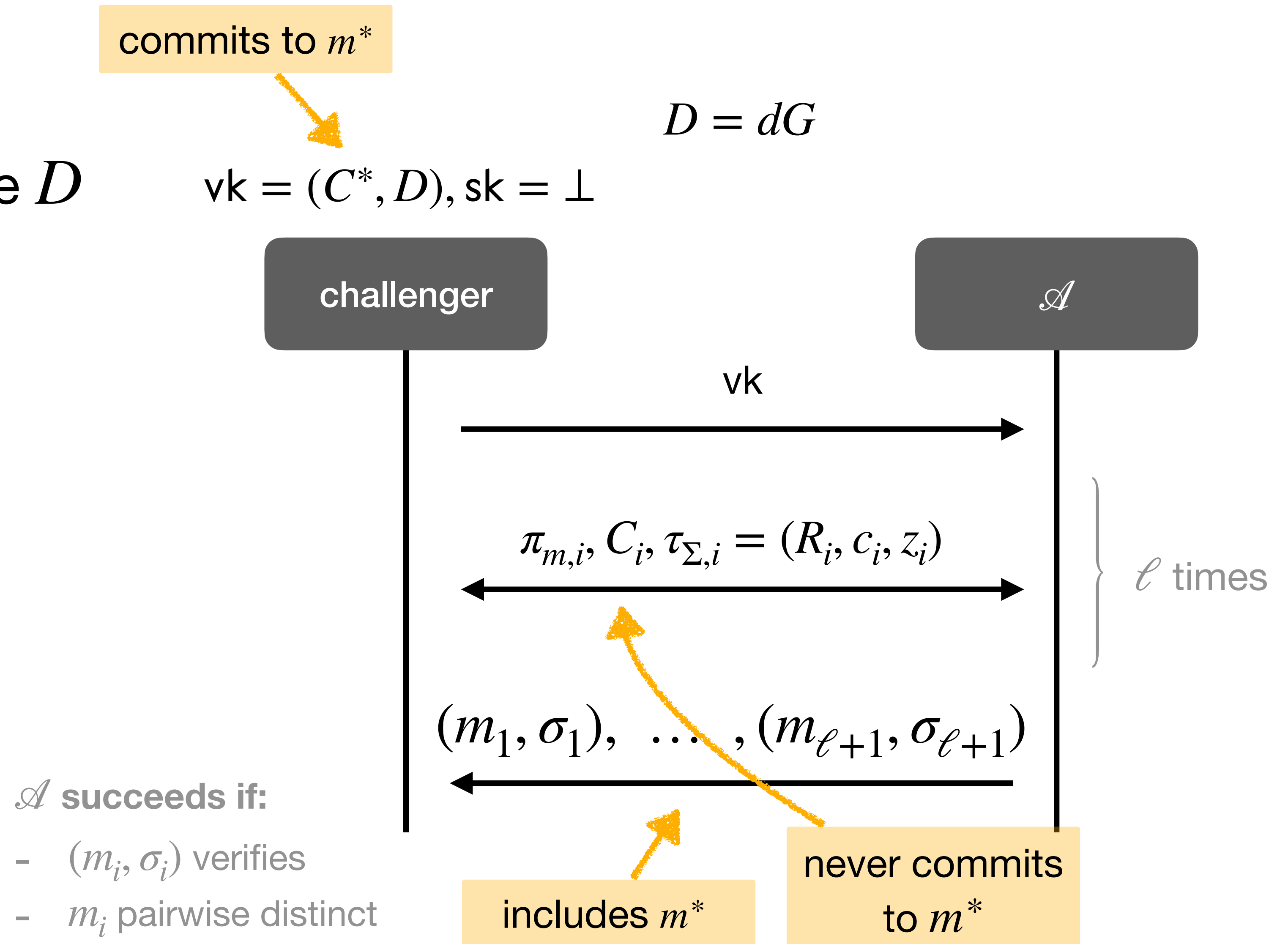
- **Step 2:** prove inequality branch honestly
 - extract opening (m_i, r_i) for C_i from proof $\pi_{m,i}$
 - compute witness to prove that $C^* - C_i$ does not commit to 0, i.e., $m_i \neq m^*$



Our Approach

Unforgeability

- **Step 3:** forget d and simulate D branch

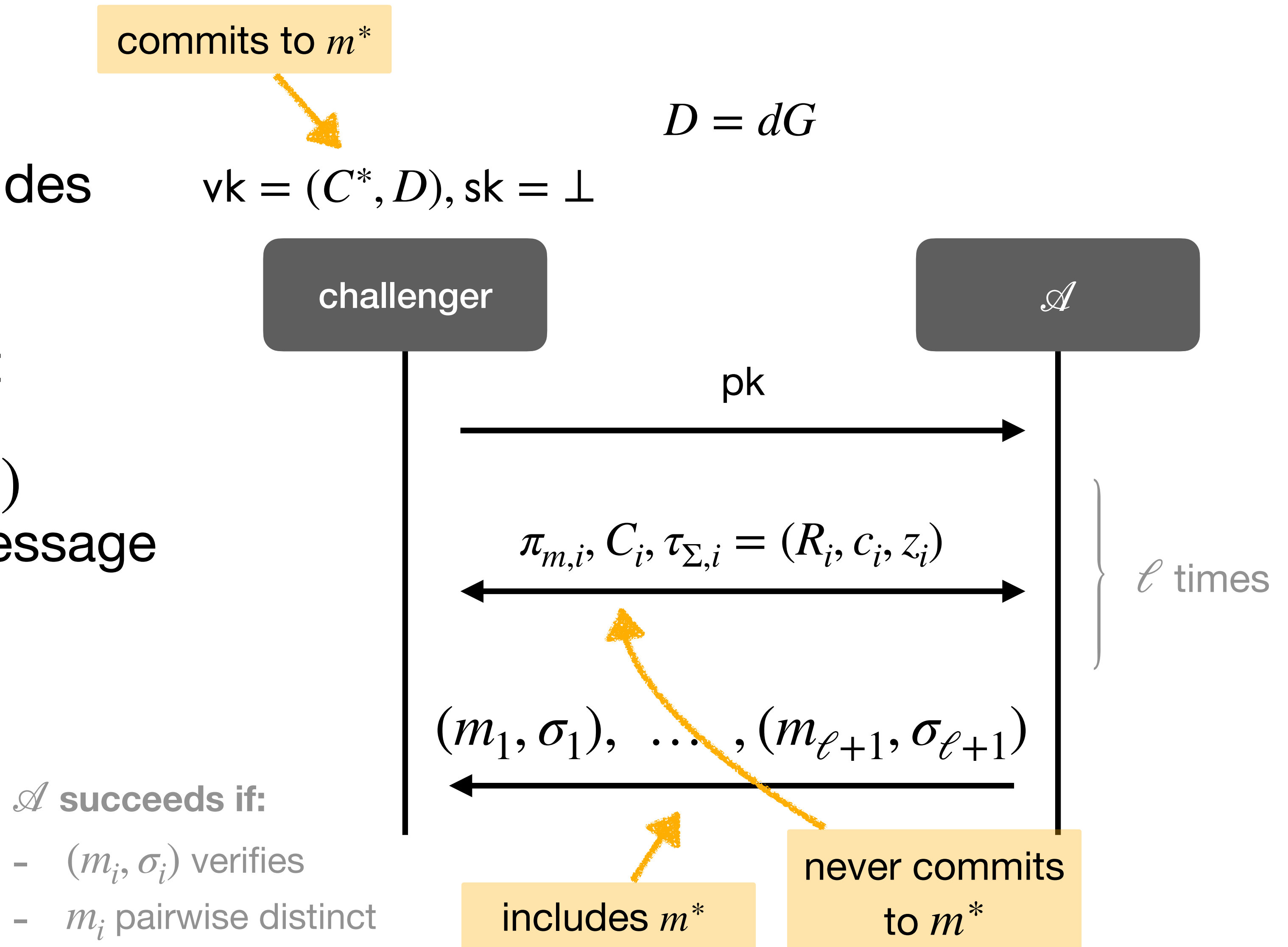


Our Approach

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*

[KR25]: claims are false as puncturing is statistical; concludes proof here



Our Approach

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*
- **Issue:** cannot embed DL challenge in a single branch
 - we might extract a witness for the branch we prove honestly

Our Approach

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*
- **Core Observation:** can embed DL challenge in both branches
 - in public parameters for Com and verification key D
 - rewind to extract *some* witness from the OR-proof

Our Approach

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*
- **CASE 1:** extract DL of D from second branch
- **CASE 2:** extract non-zero opening of $C^* - \text{Com}(m^*; 0)$ from first branch
 - does not yet allow to solve DL
 - both zero and non-zero opening together allow to compute DL



know zero
opening by setup

Summary

- **Technique:**
 - prove unforgeability via *computational* puncturing
 - rewinding-based proof by embedding DL challenge in both branches
- **Implications:**
 - blind signatures from DL in the ROM with unlimited concurrency